



ACTIVIDAD 3

TAREA MÚLTIPLE DE LA U6

Sergio García Gordo



19 DE MAYO DE 2026

U-TAD

Índice

1. Estructuración de los pasos seguidos	2
2. Memoria del desarrollo del ejercicio.....	3
2.1 Desarrollo inicial de la práctica	3
2.2 Juego lógico inicial.....	4
2.3) Traza inicial	5
2.4) Semilla ALEATORIO	5
2.5) Semilla ALEATORIO_1_3	6
3. Resultado final del programa	7
4. Capturas de la ejecución.....	8
4.1 Primera ejecución	8
4.2 Segunda ejecución	9
5. Declaración de uso de Inteligencia Artificial.....	10
6. Conclusiones y comentarios finales.....	10

1. Estructuración de los pasos seguidos

En este apartado, se detallarán los **pasos que se han seguido durante el desarrollo, basados en las directrices de la práctica con el fin de lograr la consecución de esta.**

Resumen del ejercicio: El programa genera cinco jugadas pseudoaleatorias para la máquina, codificadas con valores numéricos [1 – 3], las compara con otras cinco jugadas del jugador que se encuentran previamente almacenadas en memoria. Como resultado de la comparación, se resuelve el resultado de cada ronda.

Pasos que seguir:

- 1) **Cargar los valores iniciales de las jugadas**, tanto de la máquina como del jugador en sus respectivos registros.
- 2) Establecer lógica comparativa para saber si el **resultado de comparar ambos números** de ambos vectores es **victoria para la máquina, victoria para el jugador o empate**.
- 3) Se **actualizan los contadores de victorias, derrotas y empates**.
- 4) **Creación de las rutinas para generar números pseudoaleatorios**, las cuáles se llamarán: **ALEATORIO y ALEATORIO_1_3**
- 5) Creación de rutina **ALEATORIO**:

Información: Esta rutina implementa un generador pseudoaleatorio sencillo basado en una semilla almacenada en memoria. A partir de dicha semilla, aplica una serie de operaciones aritméticas y lógicas para obtener un nuevo valor, que se convierte en la siguiente semilla. Para escoger los valores de la semilla, comenzaremos con la semilla 23h, sumar 17 y aplicar XOR 35h.

Pasos que seguir:

Primero, se **carga la semilla**, después, guardamos los **valores en una traza** previamente creada, acto seguido, **sumamos la constante** y volvemos a **guardar el dato en la traza** y, por último, aplicamos otra **vez XOR con la constante y guardamos de nuevo en la traza**. Finalmente, **guardamos la nueva semilla**.

6.1) **Creación de una traza**, para ir registrando los valores intermedios, que se vayan generando durante el proceso con el fin de observar cómo evoluciona la secuencia generada.

6.2) **Desarrollo de la lógica del proceso**, que consiste en tomar la semilla, sumarle una constante y después aplicar una operación XOR para mezclar sus bits.

6.3) **Guardar el resultado como nueva semilla**, permitiendo seguir generando nuevos valores en llamadas posteriores.

- 6) Creación de rutina **ALEATORIO_1_3**:

Información: Esta subrutina toma el valor generado por la rutina anterior y lo ajusta para que pertenezca al rango [1-3]. Para el desarrollo del algoritmo, **reduciremos el valor dado mediante restas sucesivas** (siempre y cuando el número sea mayor o igual a 3, restándole 3 por iteración) hasta que

finalmente lo transformemos al intervalo deseado [0-2], sumándole 1 al final para que quede entre [1-3].

Pasos que seguir:

Primero, **generamos un valor base**, después lo **reducimos por restas sucesivas de 3** hasta obtener 0, 1 o 2, seguidamente **sumamos 1** y **guardamos en la traza el valor final**. Finalmente **guardamos FFh como marca de fin**.

6.1) En primer lugar, se obtiene **un número pseudoaleatorio general** a partir de la semilla anterior (ALEATORIO)

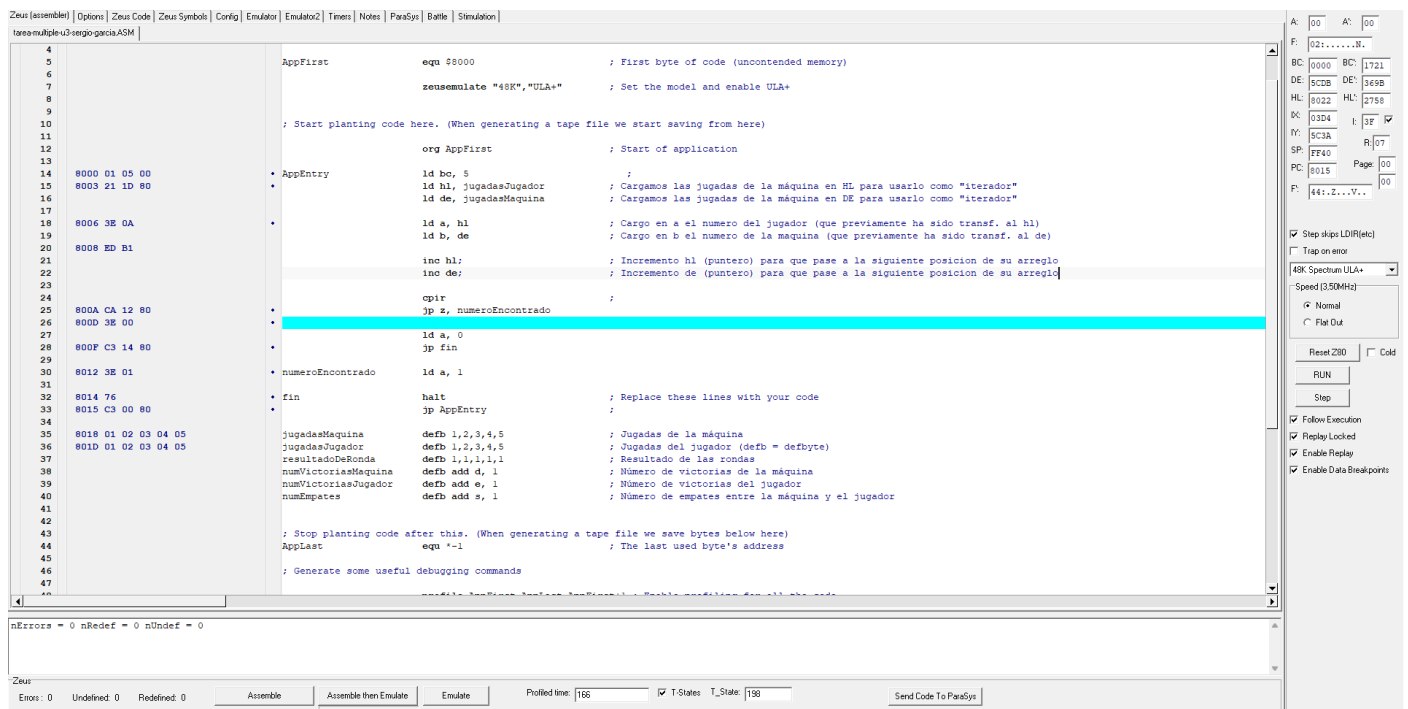
6.2) **Creación del algoritmo de reducción**, previamente explicado.

6.3) **Registra este valor final en la traza** para poder analizarlo si es necesario y añade un marcador especial que indica el final del proceso de generación de ese número.

2. Memoria del desarrollo del ejercicio

En este apartado, **se muestran capturas y explicaciones del código, durante el desarrollo de la práctica**, por lo que puede que **no tengan demasiada similitud con el resultado final**.

2.1 Desarrollo inicial de la práctica



Durante las primeras fases del desarrollo, se **escogió un ejercicio del temario**, que en un principio se creía **similar al objetivo que se pide en la práctica**, aunque finalmente se tuvo que descartar este enfoque. Podemos apreciar en la captura, cómo se desarrolló un **flujo base**, además de **arreglos hardcodedos** para intentar conseguir una ejecución mínima viable y comprobar que la lógica implementada funcionase como se requería. Todo esto teniendo en cuenta, que a **medida que avanzase el transcurso de la práctica, se debería de cambiar el arreglo de jugadasMaquina, por el valor de la semilla generada aleatoriamente**.

Problemas encontrados:

- Poca claridad en cuánto a la **estructuración del desarrollo**.
- Uso de **arreglos y variables hardcodedas**, que más adelante se reemplazarán.

- **Problemas con registros**, ya que en un principio se utilizaron registros base, en vez de *de* y *hl* para iterar sobre los arreglos, lo que causó problemas de compilación y ejecución.
- **Problemas con *defc***, ya que se **sobreutilizó para crear funciones sintácticamente erróneas** (*numVictoriasJugador*, *numVictoriasMaquina* y *numEmpates*)

2.2 Juego lógico inicial

8000 06 05	• AppEntry	ld b, 5	;
8002 21 41 80	•	ld hl, jugadasJugador	;
8005 11 3C 80	•	ld de, jugadasMaquina	;
8008 1A	• juego	ld a, (de)	;
8009 4F	•	ld c, a	;
800A 7E	•	ld a, (hl)	;
800B B9	•	cp c	;
800C 28 04	•	jr z, empate	;
800E 38 0C	•	jr c, ganaMaquina	;
8010 30 14	•	jr nc, ganaJugador	;
;-----			
8012 DD 36 00 00	• empate	ld (ix + 0), 0	;
8016 FD 36 00 00	•	ld (iy + 0), 0	;
801A 18 14	•	jr continuarBucle	;
;-----			
801C DD 36 00 00	• ganaMaquina	ld (ix + 0), 0	;
8020 FD 36 00 01	•	ld (iy + 0), 1	;
8024 18 0A	•	jr continuarBucle	;
;-----			
8026 DD 36 00 01	• ganaJugador	ld (ix + 0), 1	;
802A FD 36 00 00	•	ld (iy + 0), 0	;
802E 18 00	•	jr continuarBucle	;
;-----			
8030 23	• continuarBucle	inc hl	;
8031 13	•	inc de	;
8032 DD 23	•	inc ix; Incremento de (puntero) pa	;
8034 FD 23	•	inc iy;	;
8036 10 D0	•	djnz juego	;
;-----			
8038 76	• fin	halt	;
8039 C3 00 80	•	jp AppEntry	;
;----- Datos -----			
803C 01 02 01 02 03	jugadasMaquina	defb 1,2,1,2,3	;
8041 02 03 01 02 01	jugadasJugador	defb 2,3,1,2,1	;
8046 00 00 00 00 00	resultadosMaquina	defb 0,0,0,0,0	;
804B 00 00 00 00 00	resultadosJugador	defb 0,0,0,0,0	;

Después de haber tenido **varios problemas previos con la lógica del programa y los registros** (comentado en el punto anterior), finalmente se **desarrolló un programa que funcionaba correctamente a pesar de tener datos hardcoded**.

En la primera captura, podemos apreciar cómo se **cargan las jugadas del jugador y de la máquina** en los registros (punteros) *hl* y *de*, respectivamente. Esto funciona así ya que se **requerían de dos registros que pudiesen recorrer ambos arreglos hardcoded posición por posición**.

Más adelante, vemos cómo se **cargan los valores en los registros *a* y *c***, que se compararán por cada iteración. En función del resultado, se seleccionaba una de las tres condiciones (empate, ganaMaquina y ganaJugador), rutinas que asignan los resultados de cada partida: 1–0 o 0–1 en caso de victoria de la máquina o del jugador, respectivamente, y 0–0 en caso de empate. Al finalizar, se regresa al bucle principal, donde se incrementan los punteros para que en la siguiente iteración se comparen las siguientes posiciones de los arreglos.

Finalmente, cuando el registro ***b* fuese igual a 0, saltaría a fin**, directamente, terminando la ejecución del programa y actualizando las direcciones de memoria.

```

00803E 10 D0 76 C3 00 80 01 02 01 02 03
00804E 00 00 00 00 01 01 01 00 00 00 00
00805E 00 00 00 00 00 00 00 00 00 00 00
00806E 00 00 00 00 00 00 00 00 00 00 00

```

Imagen 1: Muestra los resultados entre la máquina (primeros 5 elementos) y el jugador (siguientes 5)

```

008034 00 00 18 00 23 13 DD 23 FD 23
008044 01 02 01 02 03 02 03 01 02 01
008054 01 00 00 00 00 00 00 00 00 00
008064 00 00 00 00 00 00 00 00 00 00

```

Imagen 2: Muestra los resultados de cada partida

2.3) Traza inicial

```

;----- Semilla aleatorio -----
semilla      defb 23h;
traza        defb 0,0,0;

```

En este apartado, se definió la **dirección a partir de la cual se inicializaría la semilla** más adelante, y la **traza** en la cuál **guardaríamos los datos de la rutina ALEATORIO**.

2.4) Semilla ALEATORIO

```

ALEATORIO      ld a, (semilla);
                ld (traza), a;
                add a, 17;
                ld (traza +1), a;
                xor 35h;

                ld (traza +2), a;
                ld (semilla), a;

```

Esta parte, hace referencia, al código necesario para la inicialización de la semilla. Cómo se puede apreciar en la captura, primero, **cargamos en el acumulador el valor de la semilla**, y luego repetimos el proceso, para **cargar en traza, el valor del acumulador (semilla)**. Después, sumamos 17 decimal al acumulador y **cargamos en la segunda posición de la traza el valor del acumulador**. Finalmente, realizamos la operación lógica XOR **entre el acumulador y la dirección de memoria 35h**.

Problema encontrado:

- En este apartado, **después de actualizar el valor de semilla con a, no hay ret (return)**, como consecuencia, la ejecución **seguía directamente con las instrucciones situadas a continuación**, lo que provocaba errores de lógica durante la ejecución del programa.

2.5) Semilla ALEATORIO_1_3

```
ALEATORIO1_3      ld (trazaAleatorio1_3), a      ;

bucle             cp 3                          ;
                  jr c, fin_reduccion           ;
                  sub 3                          ;
                  jr bucle                       ;

fin_reduccion     add a, 1                      ;
                  ld (trazaAleatorio1_3 + 1), a  ;
                  push af                       ;
                  ld a, 0FFh                    ;
                  ld (trazaAleatorio1_3 + 2), a  ;
                  pop af                        ;
                  ret                           ;

trazaAleatorio1_3 defb 0,0,0                   ;

-
```

En la captura se puede **observar la subrutina ALEATORIO1_3**, que se encarga de **aumentar el valor generado por ALEATORIO al rango 1-3**. Una vez reducido el valor mediante el bucle de restas, **en fin_reduccion se le suma 1 al acumulador y se guarda en la traza**.

Problema encontrado:

- **La rutina devolvía constantemente 255**. Esto ocurría, debido a que, cómo era **necesario guardar FFh como marca de fin en la traza**, implicaba **sobrescribir el acumulador con ese valor**, lo que hacía que el resultado correcto se perdiese. Para que eso no ocurriese, se utilizó **pushaf** antes de sobrescribir el acumulador, guardando su valor en la pila. Una vez almacenado **FFh** en la traza, se usó **popaf** para restaurar el **valor original en el acumulador, de forma que el ret final devolviese el número correcto (1, 2 o 3)** y no 255.
- **La ejecución del programa tarda mucho**, ya que *aleatorio* puede devolver un valor entre 0 y 255, y si vamos haciendo restas de 3 en 3, puede retrasar bastante la ejecución.

3. Resultado final del programa

```

AppEntry      ld b, 5                ; Cargamos inicialmente 5 en b, para que djnz tenga 5 iteraciones que va a hacer en su bucle, ya que son 5 "part:
              ld hl, jugadasJugador ; Cargamos las jugadas de la máquina en HL

              ld ix, resultadosJugador ; Utilizamos ix e iy como punteros para que vayan seleccionado la posición de los respectivos arreglos
              ld iy, resultadosMaquina ; que toca, en cada iteración del bucle.

juego         call ALEATORIO         ; Llamamos a ALEATORIO para generar un nuevo valor pseudoaleatorio a partir de la semilla actual.
              call ALEATORIOI_3      ;
              ld c, a                ; Cargo en C el número de la máquina, generado por ALEATORIOI_3.
              ld a, (hl)             ; Cargo en a el numero del jugador (que previamente ha sido transf. al hl)

              cp c                   ; Comparamos b con el acumulador (a) para ver si son iguales, uno mayor que otro etc...
              jr z, empate           ; Utilizo jr porque no hay un gran salto entre posiciones de memoria. En este caso si son iguales a y b, salto a
              jr c, ganaMaquina      ; Internamente hace A - C, cómo para este caso saldria -C (con su valor x), seria negativo y saltaria la flag C
              jr nc, ganaJugador      ; Internamente hace A - C, cómo para este caso saldria A (con su valor x), seria positivo y saltaria la flag NC

;-----
empate        ld (ix + 0), 0         ;
              ld (iy + 0), 0         ;
              jr continuarBucle      ;

ganaMaquina   ld (ix + 0), 0         ; Cargamos 0 o 1 dependiendo, en los arreglos de resultados y saltamos continuarBucle
              ld (iy + 0), 1         ;
              jr continuarBucle      ;

ganaJugador   ld (ix + 0), 1         ;
              ld (iy + 0), 0         ;
              jr continuarBucle      ;

;----- Rutina ALEATORIO -----
semilla       defb 23h              ; Definimos "semilla", como la dirección 23h.
traza         defb 0,0,0            ; Definimos una "traza" inicial, para llevar un registro de los datos.

ALEATORIO     ld a, (semilla)        ; Cargamos el valor de semilla en el acumulador (a = 23h).
              ld (traza), a          ; Luego, cargamos el valor del acumulador en el valor de traza[0] (traza[0] = a = 23h).
              add a, 17              ; Sumamos con add 17 al valor del acumulador (a = 23h + 17d).
              ld (traza+1), a        ; Cargamos el el valor del acumulador (a = 23h + 17d) en la segunda posición de la traza (traza[1]).
              xor 35h                ; Realizamos la operación lógica XOR, con 35h, comparandola con el valor de a (a XOR 35h -> (23h+17d) XOR 35h).

              ld (traza+2), a        ; Cargamos en la segunda posición de la traza (traza[2]), el valor del acumulador.
              ld (semilla), a        ; Sobreescribimos el valor en semilla, para que la proxima vez que se llame, se calcule a partir del ultimo val:
              ret                    ; Return.

;----- Rutina ALEATORIOI_3 -----
ALEATORIOI_3   ld (trazaAleatorioI_3), a ; Guardamos en la la posición 0 de trazaAleatorioI_3 el valor de a, que haria referencia a la semilla ALEATORIO,

bucle         cp 3                   ; Realizamos el bucle comparativo, que irá comparando el acumulador con 3.
              jr c, fin_reduccion    ; Si hay carry (c), es decir, es a < 3, entra en la condicion de fin y se le añadiría un 1 para que el número
              sub 3                   ; final esté entre 1 y 3. Si no, con sub 3 le quitamos 3 al acumulador, y volvemos al bucle
              jr bucle               ; para que se ejecute n veces hasta que entre en la condición especificada.

fin_reduccion  add a, 1              ; Cuando el número ya está entre 0 y 2, sumamos 1 para que quede entre 1 y 3.
              ld (trazaAleatorioI_3 + 1), a ; Cargamos el valor final (a), en la traza (posicion 1) para tener registro.
              push af                ; Guardamos a en la pila.
              ld a, 0FFh             ; Pasamos por el registro a, el valor inmediato FFh, (aunque en la pila tenemos el valor de a (1-3), que nos int
              ld (trazaAleatorioI_3 + 2), a ; Guardamos el valor de a(FFh) en la tercera posición de la traza (trazaAleatorioI_3[2]), como marca de fin.
              pop af                 ; Recuperamos el valor de a de la pila (semilla final).
              ret                    ; Hacemos un ret (return).

trazaAleatorioI_3 defb 0,0,0        ; Creamos traza especial para la semilla de ALEATORIOI_3.

;----- Bucle de piedra-papel-tijera -----
continuarBucle inc hl               ; Incremento hl (puntero) para que pase a la siguiente posicion de su arreglo
              inc ix                 ; Incremento de (puntero) para que pase a la siguiente posicion de su arreglo
              inc iy                 ;
              djnz juego             ;

fin           halt                  ; Replace these lines with your code
              jp AppEntry           ;

;----- Datos -----
jugadasJugador defb 2,3,1,2,1      ; Jugadas del jugador (defb = defbyte)

resultadosMaquina defb 0,0,0,0,0 ; Arreglos para los resultados de la máquina y del jugador.
resultadosJugador defb 0,0,0,0,0 ;

```


ALEATORIO	EQU \$803E
ALEATORIO1_3	EQU \$8052
AppEntry	EQU \$8000
AppFilename	EQU "NewFile"
AppFirst	EQU \$8000
AppLast	EQU \$8086
bucle	EQU \$8055
continuarBucle	EQU \$806D
empate	EQU \$801C
fin	EQU \$8074
fin_reduccion	EQU \$805D
ganaJugador	EQU \$8030
ganaMaquina	EQU \$8026
juego	EQU \$800D
jugadasJugador	EQU \$8078
resultadosJugador	EQU \$8082
resultadosMaquina	EQU \$807D
semilla	EQU \$803A
traza	EQU \$803B
trazaAleatoriol_3	EQU \$806A
Zeus_PC	EQU \$8000
Zeus_SP	EQU \$FF40

4. Capturas de la ejecución

4.1 Primera ejecución

ALEATORIO	EQU \$803E
ALEATORIO1_3	EQU \$8052
AppEntry	EQU \$8000
AppFilename	EQU "NewFile"
AppFirst	EQU \$8000
AppLast	EQU \$8086
bucle	EQU \$8055
continuarBucle	EQU \$806D
empate	EQU \$801C
fin	EQU \$8074
fin_reduccion	EQU \$805D
ganaJugador	EQU \$8030
ganaMaquina	EQU \$8026
juego	EQU \$800D
jugadasJugador	EQU \$8078
resultadosJugador	EQU \$8082
resultadosMaquina	EQU \$807D
semilla	EQU \$803A
traza	EQU \$803B
trazaAleatoriol_3	EQU \$806A
Zeus_PC	EQU \$8000
Zeus_SP	EQU \$FF40

```

007FDD 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
007FED 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
007FFD 00 00 00 06 05 21 78 80 DD 21 82 80 FD 21 7D 80 .....!x..!...!}.
00800D CD 3E 80 CD 52 80 4F 7E B9 28 04 38 0C 30 14 DD .>..R.O~.(.8.0..
00801D 36 00 00 FD 36 00 00 18 47 DD 36 00 00 FD 36 00 6...6...G.6...6.
00802D 01 18 3D DD 36 00 01 FD 36 00 00 18 33 7D 37 48 ..=.6...6...3}7H
00803D 7D 3A 3A 80 32 3B 80 C6 11 32 3C 80 EE 35 32 3D }::..2;...2<..52=
00804D 80 32 3A 80 C9 32 6A 80 FE 03 38 04 D6 03 18 F8 .2:...2j...8.....
00805D C6 01 32 6B 80 F5 3E FF 32 6C 80 F1 C9 7D 03 FF ..2k..>.2l...}..
00806D 23 DD 23 FD 23 10 99 76 C3 00 80 02 03 01 02 01 #.#.#..v.....
00807D 00 00 01 00 01 01 00 00 00 00 00 00 00 00 00 .....
00808D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00809D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0080AD 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0080BD 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0080CD 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0080DD 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

4.2 Segunda ejecución

ALEATORIO	EQU \$803E
ALEATORIO1_3	EQU \$8052
AppEntry	EQU \$8000
AppFilename	EQU "NewFile"
AppFirst	EQU \$8000
AppLast	EQU \$8086
bucle	EQU \$8055
continuarBucle	EQU \$806D
empate	EQU \$801C
fin	EQU \$8074
fin_reduccion	EQU \$805D
ganaJugador	EQU \$8030
ganaMaquina	EQU \$8026
juego	EQU \$800D
jugadasJugador	EQU \$8078
resultadosJugador	EQU \$8082
resultadosMaquina	EQU \$807D
semilla	EQU \$803A
traza	EQU \$803B
trazaAleatoriol_3	EQU \$806A
Zeus_PC	EQU \$8000
Zeus_SP	EQU \$FF40

```

008052 32 6A 80 FE 03 38 04 D6 03 18 F8 C6 01 32 6B 80
008062 F5 3E FF 32 6C 80 F1 C9 F5 03 FF 23 DD 23 FD 23
008072 10 99 76 C3 00 80 02 03 01 02 01 00 00 01 00 01
008082 01 01 00 00 00 00 00 00 00 00 00 00 00 00 00
008092 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0080A2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

5. Declaración de uso de Inteligencia Artificial

Durante el desarrollo, se ha utilizado la **IA Claude** para resolver dudas puntuales surgidas a lo largo del ejercicio, siendo la mayoría de ellas relativas a errores en tiempo de compilación.

Adicionalmente, en el apartado 2.5 fue necesario emplear las instrucciones *push* y *pop* para preservar el valor del acumulador en la pila. Sin ellas, al almacenar el marcador de fin *FFh* en el acumulador justo antes del ret, se retornaba 255 en lugar del resultado correcto, provocando que la máquina ganase siempre.



6. Conclusiones y comentarios finales

En conclusión, esta práctica ha resultado de gran utilidad para afianzar mis conocimientos sobre el lenguaje ensamblador. Además de eso, los errores analizados a lo largo del trabajo han contribuido a mejorar mi capacidad para estructurar problemas complejos, descomponerlos en partes más manejables y comprender la lógica interna del procesador. Sin embargo, no he logrado resolver el por qué el programa tarda tanto en finalizarse, una vez ejecutado. He mirado opciones pero ninguna ha funcionado.